



UNIVERSITÉ TOULOUSE III
PAUL SABATIER

Projet sous STM32CubeIDE - NUCLEO STM32L476RG

DÉPARTEMENT EEA – FACULTÉ SCIENCES ET INGÉNIERIE
MASTER EEA SYSTÈMES ET MICROSYSTÈMES
EMBARQUÉES (SME)

RÉALISATIONS SYSTÈMES ET MICROSYSTÈMES

COMPTE-RENDU

Student(s):

DUBOSCLARD Julian
LOZANO ORTIZ Jhonatan

Supervisor(s):

PERISSE Thierry

31 mars 2026



Table des matières

1	Introduction.	2
2	CubeIDE LED.	2
3	CubeIDE Bouton poussoir et LED.	2
4	CubeIDE Bouton poussoir et LED par interruption.	3
5	Implémentation LCD.	3
6	Implémentation du capteur ultrason.	4
7	Implémentation du capteur SHT31.	6
7.1	Protocole de communication et fonctionnement.	6
7.2	Lecture et conversion des données.	6
7.3	Implémentation logicielle sur STM32.	6
8	Cahier des charges de l'Alphabot.	7
8.1	Implémentation de l'Alphabot.	7
8.2	Projet 1 : s'immobiliser à une distance déterminée d'un obstacle.	11
8.2.1	Module Ultrasonique.	11
8.2.2	Double pont en H TB6612FNG.	11
8.2.3	Implémentation logicielle du contrôle des moteurs.	12
8.2.4	Logique principale pour l'arrêt automatique.	13
8.3	Projet 2 : suivre une ligne.	14
8.3.1	Fonctionnement général du TLC2543.	14
8.3.2	Configuration SPI et acquisition des capteurs infrarouges.	15
8.3.3	Logique suiveur de ligne.	16
8.4	Projet 3 : suivre une ligne et s'arrêter à distance avant de contourner un obstacle.	18
8.4.1	Infrarouges DSL et DSR et expenseur PC8574.	19
8.4.2	Configuration I2C et acquisition des capteurs infrarouges.	19
8.4.3	Logique pour contourner un obstacle.	20
8.5	Remarques.	21
8.5.1	Inversion des broches du microcontrôleur.	21
8.5.2	Configuration du STM32 en mode autonome.	22
8.5.3	Sensibilité des capteurs infrarouges à la lumière solaire.	22
8.5.4	Interface série pour le débogage des données.	22
8.6	Résultats.	23
8.6.1	Signaux du module ultrasonique.	23
8.6.2	Signaux de commande des moteurs.	23
8.6.3	Signaux des capteurs infrarouges IRx.	24
8.6.4	Signaux des capteurs DSL et DSR.	25
9	Conclusion.	26

1 Introduction.

L'objectif de ce document est de mener à bien plusieurs projets en quasi-autonomie, en mettant en œuvre les notions abordées en systèmes embarqués. L'absence de supervision directe nous impose également de gérer efficacement notre organisation et notre temps, afin de respecter les contraintes du cahier des charges fourni.

Dans un premier temps, nous nous familiarisons avec l'environnement de développement STM32CubeIDE ainsi qu'avec la configuration d'une carte Nucleo STM32. Ce travail inclut la prise en main des périphériques de base (LED, bouton poussoir), ainsi que la gestion des interruptions. En parallèle, une attention particulière est portée à l'analyse des schémas électroniques de systèmes existants et à l'exploitation des datasheets afin de mieux comprendre le fonctionnement des composants.

Dans une seconde phase, nous abordons différents capteurs (ultrason, température et humidité, infrarouges), ce qui nous permet d'acquérir des bases solides sur les protocoles de communication série tels que l'I²C et le SPI. Ces connaissances sont essentielles pour la suite du projet.

Le projet principal consiste en la mise en œuvre et le contrôle d'un robot AlphaBot2. Cette partie nécessite une compréhension détaillée de l'architecture matérielle et logicielle du système, ainsi que l'intégration de plusieurs fonctionnalités : contrôle des moteurs, acquisition de données capteurs et gestion de comportements autonomes. Une attention particulière est portée à la structuration du code et à l'exécution séquentielle des tâches sur le microcontrôleur.

Enfin, l'ensemble des codes développés dans le cadre de ce projet est disponible sur un dépôt GitHub accessible à l'adresse suivante : github.com/STM32TP/STM32_2026_G21. Ce dépôt regroupe les différentes implémentations réalisées tout au long du BE et permet d'assurer la traçabilité ainsi que la reproductibilité des résultats obtenus.

2 CubeIDE LED.

La première étape consiste à configurer un projet sous STM32CubeIDE afin de piloter une LED connectée au port GPIOA, sur la broche PA5. Cette broche est configurée en mode sortie (*GPIO Output*), ce qui permet de contrôler directement l'état logique de la LED. Dans le programme, la fonction `HAL_Delay()` issue de la bibliothèque HAL est utilisée afin de générer un délai d'une seconde. Cela permet d'obtenir un clignotement périodique et stable de la LED. Le code implémenté est présenté sur le dépôt GitHub correspondant.

3 CubeIDE Bouton poussoir et LED.

L'implémentation du bouton poussoir repose sur l'utilisation de la broche PA8, correspondant au bouton intégré à la carte Nucleo. Il est essentiel de configurer correctement cette entrée en définissant une résistance de type pull-up ou pull-down, afin d'éviter des états indéterminés dus au flottement du signal. Le programme développé permet de conditionner

le clignotement de la LED à l'état du bouton poussoir. Ainsi, la LED clignote uniquement lorsque le bouton est activé, illustrant une première interaction entre une entrée utilisateur et une sortie du système.

Le code implémenté est présenté sur le dépôt GitHub correspondant.

4 CubeIDE Bouton poussoir et LED par interruption.

La mise en œuvre d'un clignotement de LED à l'aide d'interruptions externes nécessite la configuration préalable d'une broche GPIO en mode EXTI (*External Interrupt*).

Ce mode permet de déclencher une interruption lors d'un événement spécifique, tel qu'un front montant, descendant ou les deux. Une fois la broche configurée, il est nécessaire d'activer l'interruption correspondante au niveau du NVIC (*Nested Vectored Interrupt Controller*), responsable de la gestion des interruptions du microcontrôleur.

Il est important de noter que l'activation du NVIC seule ne suffit pas : la configuration préalable du mode EXTI est indispensable pour permettre la génération effective des interruptions.

Cette approche permet d'améliorer la réactivité du système en évitant une surveillance continue (*polling*) de l'entrée.

5 Implémentation LCD.

L'implémentation de l'écran LCD repose sur l'utilisation du protocole de communication I²C. Pour cela, le périphérique I²C est activé dans STM32CubeIDE en configurant les broches PB8 et PB9 respectivement comme lignes SCL et SDA.

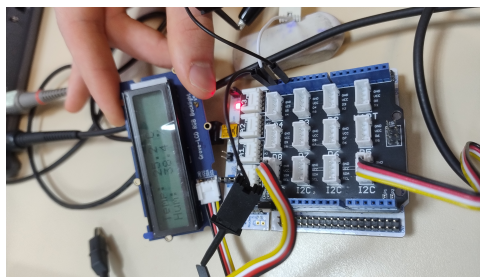


FIGURE 1 – Affichage obtenu sur l'écran LCD.

Ces broches sont configurées en mode open-drain avec des résistances de pull-up, conformément aux exigences du protocole I²C, garantissant ainsi une communication fiable sur le bus.

Le projet s'appuie sur une bibliothèque préalablement implémentée, nommée `lib_lcd.h`, qui regroupe l'ensemble des fonctions nécessaires à l'initialisation et à l'exploitation de l'écran LCD. Cette bibliothèque utilise les fonctions de la couche HAL, notamment `HAL_I2C`

`_Master_Transmit`, afin d'assurer la communication entre le microcontrôleur et le module d'affichage.

Les principales fonctionnalités offertes par cette bibliothèque incluent l'initialisation de l'écran, le positionnement du curseur, ainsi que l'affichage de chaînes de caractères (figure 1). Elle permet également de contrôler le rétroéclairage RGB.

Cette abstraction logicielle simplifie considérablement l'utilisation de l'écran LCD, tout en respectant les contraintes du protocole I²C.

6 Implémentation du capteur ultrason.

L'implémentation du capteur ultrason a débuté par une étude de sa documentation technique afin de comprendre son principe de fonctionnement. L'un des points essentiels est la nécessité de piloter précisément une broche pour générer un signal de déclenchement, puis mesurer le signal de retour associé à l'écho.

Pour cela, un signal de déclenchement d'une durée de 10 microsecondes est généré (figure 2). Cette temporisation est réalisée à l'aide d'une fonction nommée `delay_us`, permettant d'obtenir un délai précis.

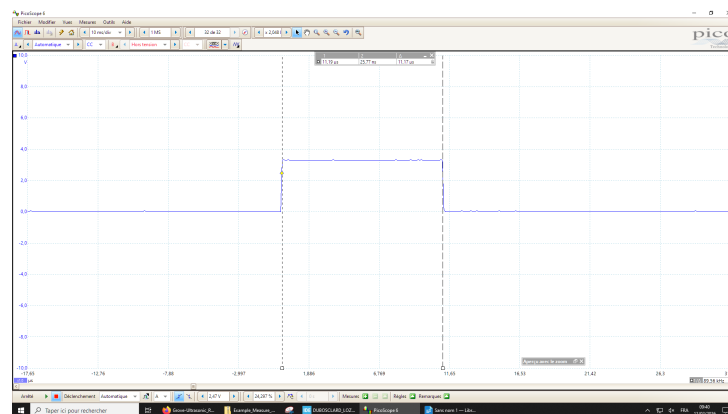


FIGURE 2 – Signal de déclenchement d'une durée de 10 microsecondes.

La fonction `delay_us` repose sur l'utilisation du compteur de cycles du processeur (DWT, *Data Watchpoint and Trace*). Elle commence par enregistrer la valeur courante du compteur DWT->CYCCNT dans une variable `startTick`. Ensuite, le nombre de cycles nécessaires pour atteindre la durée souhaitée est calculé en fonction de la fréquence du bus système (`HAL_RCC_GetHCLKFreq()`).

Le programme est ensuite maintenu en attente active (*busy-wait*) jusqu'à ce que le nombre de cycles requis soit atteint. Cette méthode permet d'obtenir une grande précision temporelle, car elle dépend directement de la fréquence du processeur, contrairement aux méthodes basées sur des interruptions.

Après l'émission du signal de déclenchement, la broche est reconfigurée afin de mesurer la durée du signal d'écho renvoyé par le capteur. Cette durée est directement liée à la distance de l'obstacle, comme illustré sur la figure 3.

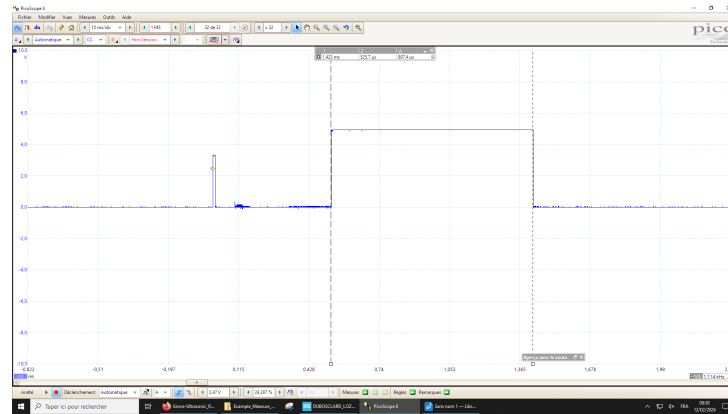


FIGURE 3 – Mesure d'une distance de 15 cm.

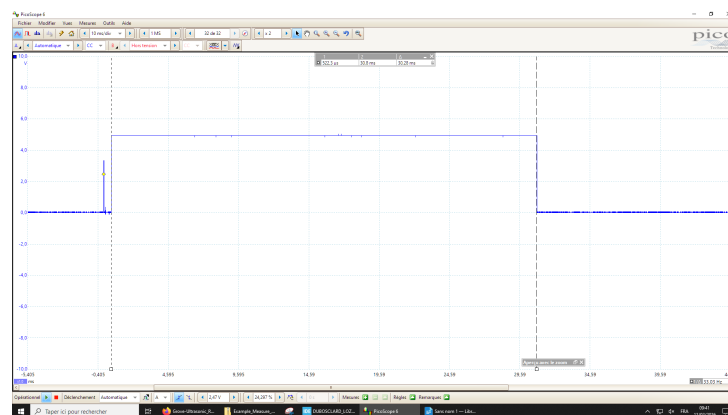


FIGURE 4 – Mesure d'une distance hors plage du capteur.

La distance est calculée à partir de la relation suivante :

$$\text{Distance [cm]} = \frac{\text{temps} \times 0.0343}{2}$$

où 0.0343 correspond à la vitesse du son en cm/ μ s. Le facteur $\frac{1}{2}$ permet de prendre en compte le trajet aller-retour de l'onde ultrasonore.

Dans notre cas, la distance mesurée correspond à la distance réelle (15 cm), ce qui valide le bon fonctionnement du capteur. Celui-ci présente une résolution de 1 cm et une plage de mesure comprise entre 3 cm et 389 cm.

Lorsque la distance mesurée dépasse les limites de fonctionnement du capteur, un mécanisme de *timeout* est utilisé afin d'éviter les interférences avec les mesures suivantes. Le signal correspondant est illustré sur la figure 4.

7 Implémentation du capteur SHT31.

Dans le cadre de ce projet, le microcontrôleur STM32 est configuré pour communiquer avec le capteur de température et d'humidité SHT31 via le protocole I²C.

7.1 Protocole de communication et fonctionnement.

Le capteur SHT31 utilise une interface I²C standard pour transmettre les données de température et d'humidité. La communication repose sur l'envoi de commandes spécifiques définies dans la documentation technique du fabricant, permettant notamment d'initialiser le capteur et d'effectuer un reset logiciel.

De plus, il est possible de configurer le mode de mesure (par exemple en haute répétabilité), de lire les données brutes de température et d'humidité, ainsi que de vérifier leur intégrité à l'aide d'un code de redondance cyclique (CRC).

7.2 Lecture et conversion des données.

L'initialisation du capteur est réalisée par l'envoi de la commande `0x30A2`, correspondant à un *soft reset*. Cette étape permet de remettre le capteur dans un état connu et de garantir un fonctionnement fiable.

La lecture des mesures est effectuée en envoyant la commande `0x2400`, qui déclenche une acquisition en mode haute répétabilité. Après un délai d'environ 15 ms, nécessaire à la conversion, le microcontrôleur récupère 6 octets de données via le protocole I²C.

Ces données se décomposent comme suit :

- 2 octets pour la température brute (`rawT`),
- 1 octet de CRC associé (non exploité dans cette implémentation),
- 2 octets pour l'humidité brute (`rawH`),
- 1 octet de CRC associé.

Les valeurs brutes sont ensuite converties en unités physiques à l'aide des relations fournies par le fabricant :

$$\begin{aligned}\text{Température [}^{\circ}\text{C]} &= -45 + 175 \times \frac{\text{rawT}}{65535} \\ \text{Humidité relative [\%]} &= 100 \times \frac{\text{rawH}}{65535}\end{aligned}$$

7.3 Implémentation logicielle sur STM32.

L'implémentation logicielle repose sur deux fonctions principales définies dans le fichier `sht31.c` :

- `SHT31_Init(I2C_HandleTypeDef *hi2c)`

— SHT31_Read(I2C_HandleTypeDef *hi2c, float *temperature, float *humidity)

La fonction SHT31_Init réalise le reset logiciel du capteur, tandis que la fonction SHT31_Read permet d'envoyer la commande de mesure, de récupérer les données et d'effectuer leur conversion.

Dans le fichier main.c, les valeurs mesurées sont affichées sur un écran LCD via le bus I²C et combinées avec la mesure de distance issue du capteur ultrasonique. Les résultats sont affichés sous une forme lisible, incluant une partie entière et une partie décimale.

8 Cahier des charges de l'Alphabot.

Notre cahier des charges est composé de trois étapes. Premièrement, le robot doit être capable de s'immobiliser à une distance précise de 10 cm d'un obstacle. Deuxièmement, il doit effectuer un suivi de ligne. Enfin, le dernier objectif consiste à intégrer une fonction pour éviter un obstacle : après s'être arrêté devant l'obstacle, le robot doit pouvoir le contourner pour reprendre sa trajectoire.

Les codes implémentés sont présentés sur le dépôt GitHub correspondant.

8.1 Implémentation de l'Alphabot.

Dans un premier temps, nous avons étudié des documents techniques tels que :

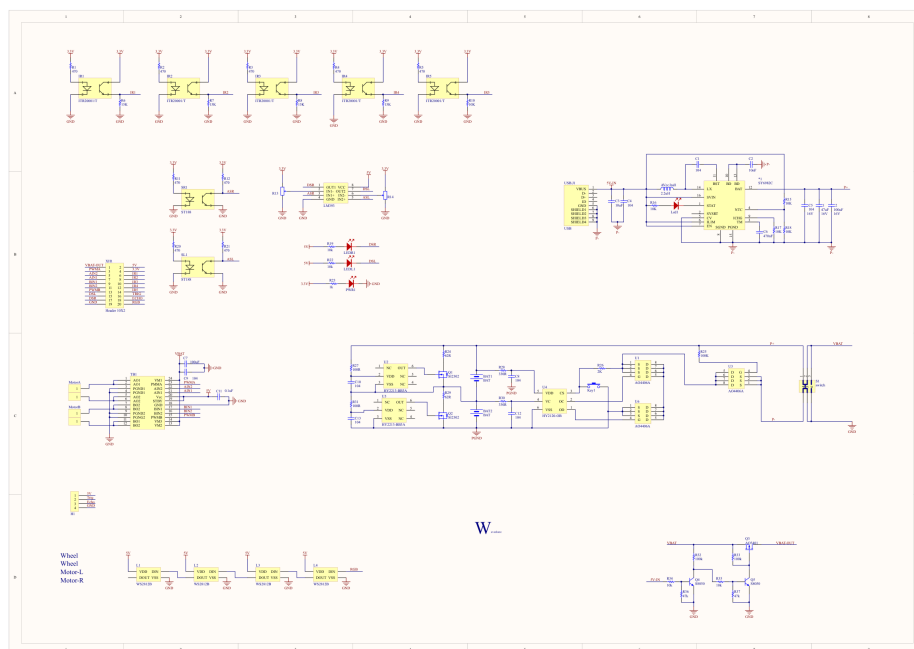


FIGURE 5 – Schema électrique AlphaBot2-Base.

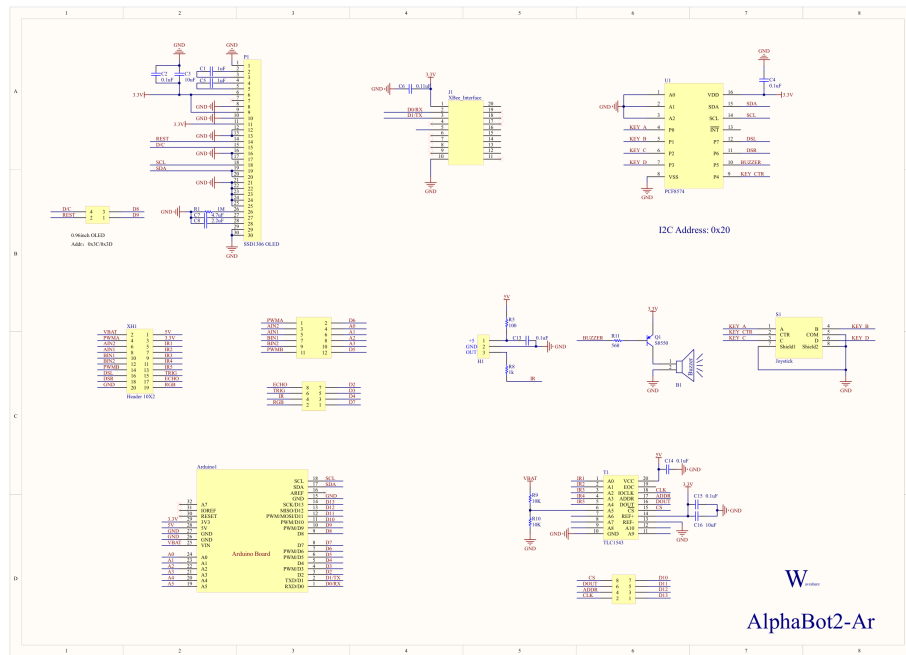


FIGURE 6 – Schema électrique AlphaBot2-AR.

Les figures 6 et 5 permettent de comprendre l'architecture du système, notamment le fonctionnement des portes logiques utilisées par les capteurs, les composants de puissance, ainsi que les ports utilisés pour la communication entre l'AlphaBot2 et la carte STM32.

Afin de faciliter l'identification des différents composants du système, nous nous appuyons sur des schémas structurels disponibles dans la documentation. Ces représentations permettent de localiser rapidement les composants et d'accéder à leurs fiches techniques pour une étude approfondie. Les figures 7 et 8 présentent l'architecture matérielle du robot AlphaBot2, qui repose sur une conception modulaire composée de deux cartes électroniques distinctes interconnectées par un bus.

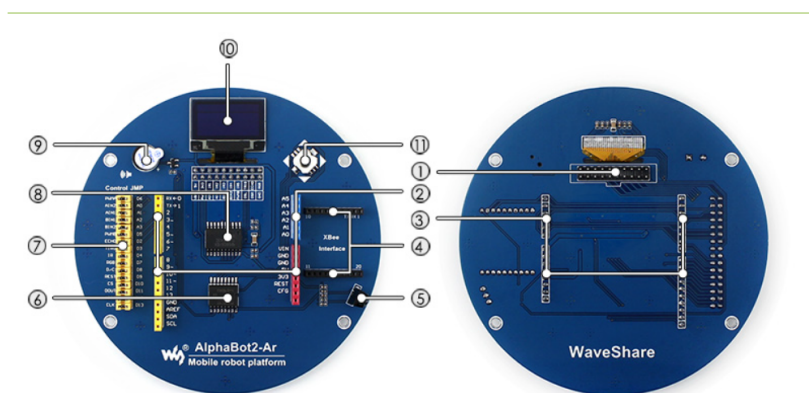


FIGURE 7 – Structure de l'Alphabot2-AR.

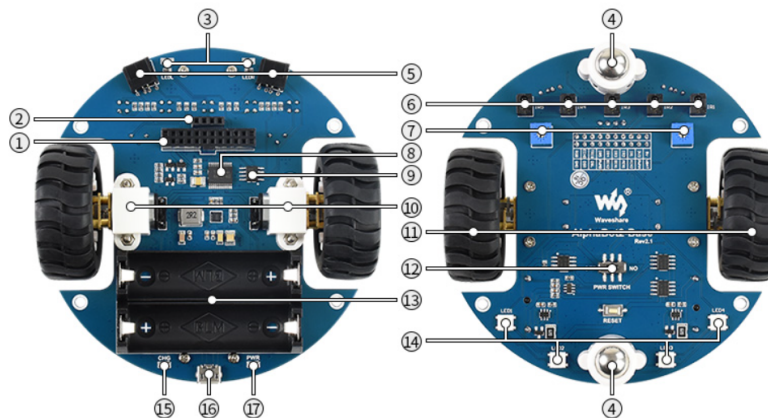


FIGURE 8 – Structure de l'Alphabot2 base.

Les plans ci-dessus montrent la structure de l'AlphaBot2 et, via les sites officiels, on trouve les noms et les détails des composants à étudier.

La carte inférieure, appelée AlphaBot2-Base, regroupe l'ensemble des éléments liés à la perception et à la puissance. Elle intègre notamment les capteurs infrarouges de suivi de ligne (IR1 à IR5) à sortie analogique, ainsi que les capteurs d'évitement d'obstacles (DSL et DSR). Elle dispose également d'une interface pour module ultrasonique directement accessible via des connecteurs soudés. En plus de la partie capteurs, cette carte embarque les composants nécessaires à la gestion de l'énergie, tels que le circuit de charge de la batterie via le port USB 5V, ainsi que les éléments de puissance pour l'actionnement, incluant les moteurs N20 et le driver TB6612FNG (double pont en H) pour leur commande.

La carte supérieure, AlphaBot2-AR, joue le rôle d'interface avec le microcontrôleur. Elle fournit les connecteurs nécessaires à l'intégration d'une carte de contrôle (initialement de type Arduino), ainsi que des interfaces d'extension. Elle intègre également des composants essentiels pour le traitement des signaux, notamment le convertisseur analogique-numérique TLC2543, utilisé pour l'acquisition des signaux des capteurs infrarouges, ainsi que l'expandeur d'entrées/sorties PC8574, permettant de récupérer les signaux des capteurs DSL et DSR, mais aussi de gérer le buzzer.

Initialement, dans le cadre de ce projet, un shield additionnel Vita Science est utilisé afin d'assurer l'adaptation entre le microcontrôleur STM32L476RG Nucleo et les connecteurs de la carte AlphaBot2-AR. Cette solution permet une correspondance directe des broches et simplifie l'intégration matérielle sans modification du circuit existant.

Parmi les principaux composants identifiés pour notre projet, on peut citer :

- (2 base) Interface pour module ultrasonique
- (8 base) Driver de moteurs TB6612FNG (double pont en H)
- (10 base) Moteurs N20 avec réducteur (rapport 1 :30, 6V, 600 RPM)
- (16 base) Port USB 5V pour l'alimentation et la recharge de la batterie
- (2 AR) Connecteur d'extension type Arduino (pour shields)

- (6 AR) Expansur d'entrées/sorties PC8574 (interface I²C)
- (8 AR) Convertisseur analogique-numérique TLC2543 (résolution 12 bits)

Le tableau 1 présente la correspondance entre les broches du microcontrôleur STM32 et les différents modules du robot AlphaBot2-AR. On observe que le pilotage des moteurs est assuré par le driver TB6612FNG, qui utilise plusieurs broches GPIO pour le contrôle de direction (AIN1, AIN2, BIN1, BIN2) ainsi que des entrées PWM (PWMA, PWMB) pour la gestion de la vitesse. Le convertisseur analogique-numérique externe TLC2543 est connecté via une interface de type SPI simplifiée (CS, CLK, DOUT, ADDR), permettant la lecture des capteurs infrarouges du suiveur de ligne (IR1 à IR5). Par ailleurs, l'expansur d'entrées/sorties PC8574 communique avec le microcontrôleur à travers le bus I²C (SDA, SCL), permettant la lecture des capteurs infrarouges DSR et DSL. Enfin, le module ultrasonique est relié à deux broches GPIO dédiées (TRIG et ECHO), utilisées respectivement pour l'émission et la réception du signal ultrasonore afin de mesurer la distance.

Broche STM32	Module / Interface	Signal
PB10	Driver TB6612FNG	PWMA
PB4		PWMB
PA0		AIN2
PA1		AIN1
PA4		BIN1
PB0		BIN2
PB6	ADC TLC2543	CS
PA6		DOUT
PA7		ADDR
PA5		CLK
PB9	Expansur PC8574	SDA
PB8		SCL
PA10	Module ultrason	ECHO
PB3		TRIG

TABLE 1 – Correspondance des broches entre le STM32L476RG et les modules de l'AlphaBot2-AR

Dans le cadre de notre projet, nous présentons l'architecture des connexions du système, en nous concentrant uniquement sur les signaux et modules réellement utilisés. Bien que l'AlphaBot2-AR offre de nombreuses interfaces et fonctionnalités supplémentaires, notre étude porte sur les éléments pertinents pour le contrôle via le microcontrôleur STM32.

La figure 9 illustre les principales connexions entre la carte AlphaBot2-AR et notre STM32. Comme nous l'avons vu dans le tableau 1, cette représentation permet d'identifier clairement les broches STM32 associées à chaque module actif du robot, facilitant la compréhension du câblage, le flux des données et du traitement des signaux.

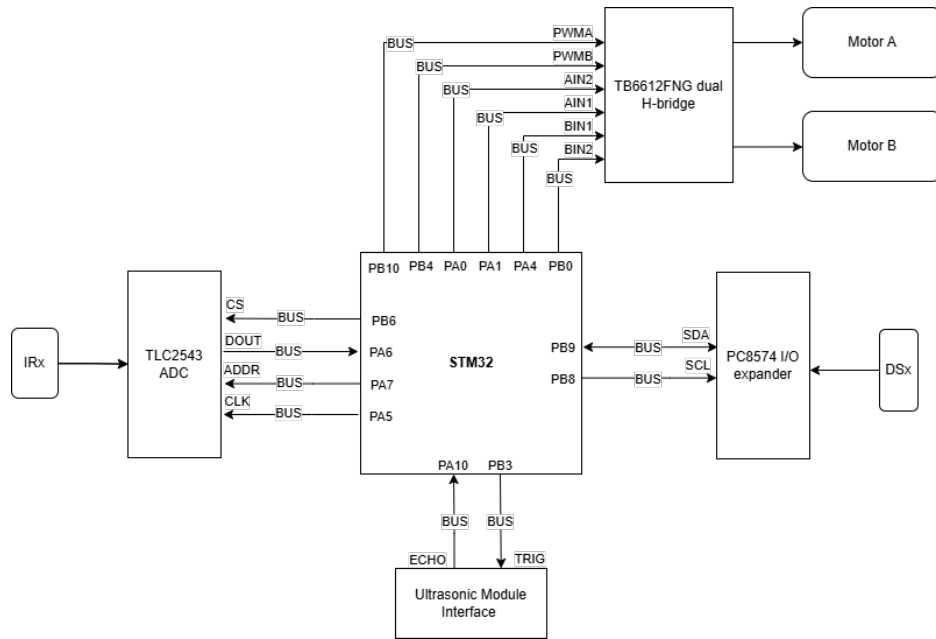


FIGURE 9 – Architecture de notre système.

8.2 Projet 1 : s'immobiliser à une distance déterminée d'un obstacle.

Pour le Projet 1, l'étude du capteur avait été réalisée préalablement, ce qui nous a permis d'avancer plus rapidement.

L'objectif était d'utiliser les deux moteurs pour que le robot se déplace et s'arrête automatiquement en présence d'un objet situé à 10 cm ou moins. Cet objectif a été pleinement atteint grâce à l'élaboration d'un code spécifique gérant cet état de fonctionnement.

8.2.1 Module Ultrasonique.

Le capteur utilisé est le même que celui étudié précédemment, à une différence près : il possède 4 broches (VCC, GND, ECHO et TRIG) au lieu de 3. Cette configuration permet de le piloter via deux broches distinctes (PA10 et PB3), évitant ainsi d'avoir à modifier les propriétés d'une broche (entrée/sortie) pendant l'envoi ou la réception des données. Les exemples et les explications des signaux du module ultrasonique sont situés à la partie 8.6.1.

8.2.2 Double pont en H TB6612FNG.

Le TB6612FNG est un contrôleur et pilote de moteur permettant d'assurer la communication entre le STM32 et les deux moteurs, en leur fournissant à chacun un signal PWM adapté, comme illustré sur la figure 9. En pratique, le microcontrôleur envoie des signaux logiques (HIGH/LOW et PWM), et le driver convertit ces signaux en puissance électrique adaptée aux moteurs.

Le principe fondamental du pont en H repose sur la capacité à inverser le sens du courant traversant le moteur. Cette inversion permet de contrôler le sens de rotation. En complément, la vitesse du moteur est ajustée à l'aide d'un signal de modulation de largeur d'impulsion (PWM), appliqué à l'entrée d'activation du driver. Ainsi, le système permet un contrôle simultané du sens et de la vitesse de rotation.

Le fonctionnement du driver est défini par une table de vérité appelée *H-SW Control Function*, qui établit la correspondance entre les entrées logiques et l'état du moteur. Les deux entrées principales, notées IN1 et IN2, déterminent le mode de fonctionnement, tandis que le signal PWM module la vitesse lorsque le moteur est actif.

IN1	IN2	PWM	État du moteur
0	0	X	Arrêt (roue libre)
1	0	PWM	Rotation avant
0	1	PWM	Rotation arrière
1	1	X	Freinage

TABLE 2 – Table de contrôle H-SW du driver moteur

Lorsque les deux entrées IN1 et IN2 sont à l'état bas, le moteur n'est pas alimenté et se trouve en roue libre, ce qui signifie qu'il peut continuer à tourner sous l'effet de son inertie. Lorsque IN1 est à l'état haut et IN2 à l'état bas, le courant circule dans un sens donné, entraînant une rotation dans le sens direct. Inversement, lorsque IN1 est à l'état bas et IN2 à l'état haut, le sens du courant est inversé, ce qui provoque une rotation en sens opposé. De la même manière, lorsque les deux entrées sont simultanément à l'état haut, les bornes du moteur sont court-circuitées, ce qui induit un freinage électrique. Ce mode permet un arrêt rapide du moteur, contrairement à l'arrêt en roue libre. Le signal PWM, quant à lui, agit uniquement lorsque le moteur est en rotation, en modulant la tension moyenne appliquée et donc la vitesse de rotation.

Ce mécanisme de contrôle permet une gestion précise des déplacements du robot. En commandant indépendamment les deux moteurs, il est possible de réaliser des mouvements variés tels que l'avance, le recul ou les rotations, en ajustant à la fois le sens et la vitesse de chaque roue.

8.2.3 Implémentation logicielle du contrôle des moteurs.

L'implémentation logicielle du contrôle des moteurs repose directement sur les principes décrits précédemment, notamment la table de contrôle *H-SW Control Function*. Le code développé permet de piloter indépendamment les deux moteurs du robot en agissant sur les entrées logiques du driver ainsi que sur les signaux PWM.

L'initialisation des moteurs est réalisée par la fonction "*Motor_Init*", qui active les signaux PWM sur deux canaux de temporisateurs matériels. Ces signaux correspondent aux entrées d'activation du driver (PWMA et PWMB) et permettent de moduler la vitesse des moteurs. L'utilisation de temporisateurs matériels garantit une génération stable et précise du signal PWM.

La fonction "*Motor_Stop*" permet de stopper les moteurs en imposant un rapport cyclique nul sur les deux signaux PWM. Ce comportement correspond, du point de vue matériel, à une absence d'alimentation effective du moteur, ce qui se rapproche du mode d'arrêt en roue libre décrit dans la table 2.

La fonction principale "*Moteur_Set*" assure le contrôle du sens de rotation et de la vitesse pour chaque moteur. Les paramètres **gauche** et **droite** représentent respectivement les consignes de vitesse pour les moteurs gauche et droit. Le signe de ces valeurs détermine le sens de rotation, tandis que leur valeur absolue correspond au rapport cyclique du PWM.

Lorsque la consigne est positive, le code positionne les entrées logiques de manière à obtenir une configuration de type ($IN1 = 1, IN2 = 0$), ce qui correspond, selon la table 2, à une rotation dans le sens direct. En revanche, lorsque la consigne est négative, les entrées sont inversées ($IN1 = 0, IN2 = 1$), ce qui produit une rotation en sens inverse.

Après la détermination du sens de rotation, la valeur absolue de la consigne est utilisée pour définir le rapport cyclique du signal PWM via les registres de comparaison des temporisateurs. Une saturation est également appliquée afin de limiter la valeur maximale du PWM, garantissant ainsi un fonctionnement sûr du système.

Enfin, le contrôle indépendant des deux moteurs permet de générer différents types de comportements du robot, tels que l'avance, le recul ou les rotations, en combinant de manière appropriée les consignes appliquées à chaque moteur.

8.2.4 Logique principale pour l'arrêt automatique.

Pour que le robot s'arrête automatiquement à une distance déterminée d'un obstacle, il est nécessaire d'intégrer dans le programme principal une boucle qui combine la lecture du capteur ultrasonique et le contrôle des moteurs.

La fonction "*Ultrasonic_Read*" permet d'obtenir en temps réel la distance séparant le robot de l'objet le plus proche. Le code vérifie ensuite la validité de la mesure ; si la distance retournée est négative, elle est remplacée par zéro afin de prévenir toute erreur de calcul.

La logique de contrôle peut être décrite par l'algorithme suivant (figure 10) :

1. Lire la distance via le capteur ultrasonique : `distance = Ultrasonic_Read()`
2. Vérifier la validité de la mesure : `if(distance < 0)`
3. Comparer la distance avec la valeur seuil (10 cm) :
 - Si `distance ≤ 10` cm, appeler "*Motor_Stop*()" pour arrêter immédiatement les moteurs. Cette action correspond au mode d'arrêt décrit précédemment dans la table 2 *H-SW Control Function* ($IN1 = IN2 = 0$, roue libre).
 - Sinon, continuer à déplacer le robot en appliquant une consigne de vitesse négative pour avancer vers l'obstacle : `Motor_Set(-200, -200)`. Les valeurs négatives inversent le sens de rotation selon la logique du pont en H.

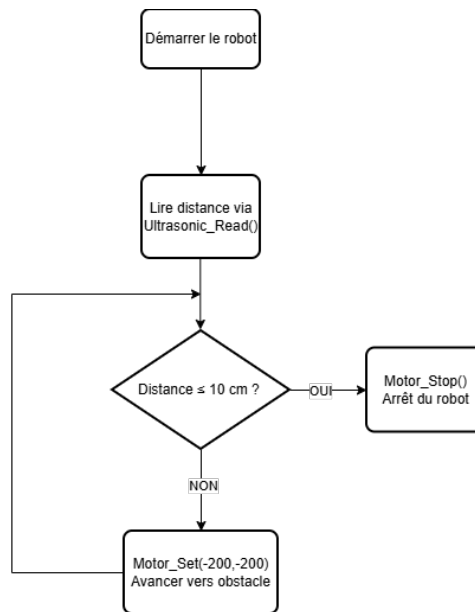


FIGURE 10 – Diagramme de flux pour l'arrêt automatique du robot à distance prédéfinie.

Cette approche simple mais efficace permet au robot de se déplacer de manière autonome tout en s'arrêtant à une distance prédéterminée d'un obstacle. La combinaison de la lecture du capteur ultrasonique et du contrôle des moteurs via le driver TB6612FNG illustre parfaitement l'intégration matérielle et logicielle nécessaire pour la réalisation de comportements automatiques sur un robot mobile.

8.3 Projet 2 : suivre une ligne.

L'objectif du Projet 2 était de permettre au robot de suivre une ligne de manière autonome. Pour ce faire, nous avons utilisé des capteurs infrarouges de suivi de ligne. Ces derniers permettent de mesurer des valeurs de réflexion que nous traitons par code en définissant un seuil, permettant au robot de distinguer la ligne à suivre du reste de la surface, dont la valeur est inférieure au seuil choisi. Ce seuil a été ajusté suite aux différents tests effectués sur de nombreux circuits différents. Les infrarouges suiveur de ligne sont testés via un code test développé par nous, afin de pouvoir observer les valeurs en temps réel et d'effectuer des tests de fonctionnements ainsi qu'obtenir les premières valeurs de seuil par exemple.

Afin de garantir un suivi fluide et précis pour le robot, nous avons implémenté un correcteur PD (Proportionnel, Dérivé). Les paramètres du PD ont été calibrés en fonction de différentes variables, comme la vitesse de déplacement choisie pour l'AlphaBot2. Cet objectif a été intégralement atteint.

8.3.1 Fonctionnement général du TLC2543.

Le TLC2543 est un convertisseur analogique-numérique (CAN) de 12 bits à 11 entrées analogiques multiplexées fabriqué par Texas Instruments. Il permet la conversion de signaux analogiques en signaux numériques pour des applications de contrôle et de mesure.

Les cinq capteurs infrarouges (numérotés de 1 à 5) sont reliés au TLC2543 visible sur la figure 9 ci-dessus. Ce composant communique avec le microcontrôleur via le protocole SPI.

Les principaux signaux utilisés par ce composant sont :

- **CLK (Clock)** : Horloge utilisée pour synchroniser la transmission des données et la conversion analogique-numérique. Chaque impulsion d'horloge fait avancer le registre interne pour le séquençement des données.
- **ADDR (Adresse)** : Permet de sélectionner quel canal d'entrée analogique sera converti. Le TLC2543 dispose de plusieurs entrées analogiques multiplexées, et ce signal choisit l'entrée active.
- **DOUT (Data Out)** : Sortie série des données numériques résultant de la conversion analogique-numérique. Les données sont transmises bit par bit en synchronisation avec le signal CLK.
- **CS (Chip Select)** : Signal de sélection du composant. Lorsqu'il est actif (niveau bas), le TLC2543 est activé pour la conversion et la transmission des données. Lorsqu'il est inactif (niveau haut), le composant ignore les signaux d'entrée.

Le TLC2543 fonctionne selon un mécanisme de conversion successive-approximation avec pipeline intégré. Lorsqu'une transition de CS passe de haut à bas, le composant est activé et le registre d'adresse reçoit les quatre bits provenant de ADDR pour sélectionner le canal d'entrée analogique à convertir. Dès le quatrième front descendant de CLK, la tension analogique du canal sélectionné commence à charger le réseau de condensateurs interne, tandis que les huit bits du résultat de la conversion précédente sont décalés sur DOUT en phase avec CLK. Ce processus permet de chevaucher la conversion actuelle avec la lecture des données précédentes, améliorant ainsi le débit du système. Une fois la conversion terminée, le signal EOC passe à l'état bas pour indiquer que les données du cycle en cours sont prêtes. Il convient de préciser que dans notre cas, le signal EOC n'est connecté à aucune partie du circuit, donc CS est utilisé pour déterminer le début et la fin d'un échange de données. Grâce à cette architecture pipeline, le TLC2543 permet une acquisition rapide et continue de signaux analogiques multiples tout en minimisant le temps mort entre les conversions.

Cependant, la mise en œuvre de la communication SPI nous a contraints à retirer le shield. En effet, un problème de broche entre la carte STM32 et l'AlphaBot2-AR empêchait la communication lorsque le shield était présent. Pour remplacer le shield, nous avons remplacé toutes les liaisons des broches utilisées par le shield par des câbles afin d'obtenir une meilleure mobilité des choix des broches de communication. Ce problème est détaillé dans la section 8.5.1.

8.3.2 Configuration SPI et acquisition des capteurs infrarouges.

La communication se fait par le protocole SPI configuré dans CubeIDE avec les paramètres suivants :

- **Mode SPI** : Maître (SPI_MODE_MASTER)
- **Direction** : 2 lignes (full duplex)

- **Taille des données** : 12 bits (SPI_DATASIZE_12BIT)
- **Polarité de l'horloge** : basse (SPI_POLARITY_LOW)
- **Phase de l'horloge** : front montant (SPI_PHASE_1EDGE)
- **NSS** : logiciel (SPI_NSS_SOFT)
- **Premier bit transmis** : MSB (SPI_FIRSTBIT_MSB)
- **Prescaler** : 64 (SPI_BAUDRATEPRESCALER_64)

Ces paramètres garantissent que le microcontrôleur peut piloter correctement le TLC2543 étant donné que la polarité et la phase de l'horloge sont compatibles avec les exigences du composant, et la taille de données de 12 bits permet d'envoyer les 4 bits de commande d'adresse suivis de 8 bits de décalage pour récupérer la conversion précédente.

Lorsqu'un canal est demandé avec la fonction `"HAL_GPIO_WritePin"`, le signal CS passe à l'état bas et le registre interne reçoit le canal via ADDR (en utilisant `"HAL_SPI_TransmitReceive"`). En synchronisation avec CLK, la conversion en cours s'effectue tandis que les données de la conversion précédente sont décalées sur DOUT. Ainsi, pour obtenir les lectures correctes de plusieurs capteurs, le code utilise une séquence décalée. Chaque appel à `"TR_TLC2543_ReadChannel"` lance la conversion du canal courant et retourne la valeur du cycle précédent. Les résultats sont ensuite stockés dans un tableau, garantissant un flux continu de données sans temps mort.

Les données acquises par le TLC2543 sont ensuite transmises via l'interface série du microcontrôleur et affichées sur un terminal comme PuTTY (figure 17), en utilisant le driver série IDUINO. Cela permet de visualiser en temps réel les valeurs des capteurs infrarouges pour le débogage et l'analyse.

8.3.3 Logique suiveur de ligne.

L'acquisition des données des capteurs infrarouges via le TLC2543 permet de calculer la position de la ligne et d'appliquer un contrôle proportionnel-dérivé (PD) pour piloter les moteurs du robot. La fonction `"LineControl_PD"` implémente un algorithme de suivi de ligne (figure 11) basé sur un régulateur proportionnel-dérivé (PD). Elle utilise les valeurs de cinq capteurs infrarouges pour estimer la position de la ligne et ajuster la vitesse des moteurs du robot.

Chaque capteur est associé à un poids représentant sa position physique :

```
int weights[5] = {0,1000,2000,3000,4000};
```

Une boucle parcourt les capteurs afin de calculer un barycentre ("centre de masse") :

```
for(int i=0;i<5;i++)
{
    int value = THRESHOLD - s[i];

    if(value < 0){
        value = 0;
    }
}
```

```
    if(value > 1400){  
        value = 1400;  
    }  
  
    weighted_sum += (long)value * weights[i];  
    sum += value;  
}
```

La variable "*value*" représente l'intensité de détection de la ligne. Les valeurs sont saturées pour limiter le bruit et éviter qu'un capteur domine excessivement.

La position de la ligne est ensuite calculée par :

```
int position = weighted_sum/sum;
```

Si aucun capteur ne détecte la ligne ("*sum* == 0"), le robot effectue une rotation dans la direction de la dernière erreur connue :

```
if(sum == 0)  
{  
    if(last_error > 0){  
        Moteur_Set(-200,200);  
    }else{  
        Moteur_Set(200,-200);  
    }  
    return;  
}
```

L'erreur est définie comme l'écart entre la position actuelle et le centre :

```
float error = (position-2000)/1000.0f;
```

Une erreur négative indique que la ligne est à gauche, et une erreur positive qu'elle est à droite.

Le signal de commande est calculé avec un contrôleur PD :

```
float control = KP*error + KD*(error-last_error);
```

- Le terme proportionnel (P) corrige l'écart à la ligne.
- Le terme dérivatif (D) anticipe les variations pour réduire les oscillations.

L'erreur précédente est ensuite mise à jour :

```
last_error = error;
```

Le signal de commande est limité pour éviter des corrections trop brusques :

```
if(control > 400){
    control = 400;
}
if(control < -400){
    control = -400;
}
```

Les vitesses des moteurs sont ajustées en fonction du signal de contrôle :

```
int left = BASE_SPEED + control;
int right = BASE_SPEED - control;
```

```
Moteur_Set(-left,-right);
```

Cette stratégie permet au robot de corriger sa trajectoire en continu :

- Si la ligne est à droite, le robot tourne à droite.
- Si la ligne est à gauche, le robot tourne à gauche.

De cette manière, le contrôleur PD permet un suivi de ligne fluide et stable. Contrairement à une logique discrète, cette approche repose sur une estimation continue de la position, ce qui améliore la précision et réduit les oscillations du robot.

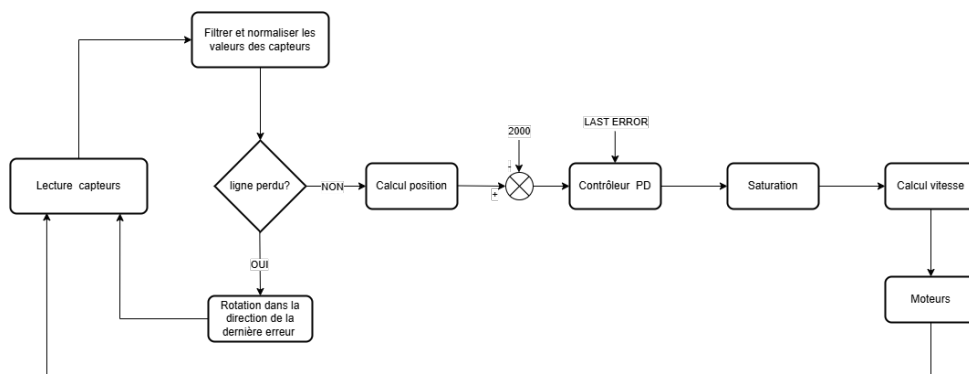


FIGURE 11 – Schéma de contrôle du suivi de ligne avec régulateur PD.

8.4 Projet 3 : suivre une ligne et s'arrêter à distance avant de contourner un obstacle.

Lors de ce projet, nous avons procédé par étapes. La première étape a consisté à arrêter le robot devant l'obstacle. Ensuite, nous avons fait en sorte que le robot s'arrête puis effectue un demi-tour afin de repartir dans la direction opposée. Dans une troisième phase, après de nombreuses tentatives, nous avons réussi à faire contourner l'obstacle au robot en décrivant un arc de cercle après l'arrêt. Enfin, une fonctionnalité supplémentaire a été ajoutée : le robot effectue un demi-tour à la fin de la ligne afin de rester sur le parcours défini.

La réalisation de ce projet a nécessité l'utilisation simultanée de plusieurs capteurs. Dans un premier temps, nous avons utilisé les cinq capteurs infrarouges (IR) de suivi de ligne avec le capteur à ultrasons pour gérer l'arrêt devant l'obstacle. Par la suite, nous avons employé les deux capteurs IR frontaux de l'AlphaBot2-Base. Ces derniers permettent de détecter les obstacles de manière plus précise et de s'assurer que le robot n'entre pas en collision avec des objets sur son chemin lorsqu'il contourne.

La gestion simultanée de l'ensemble des capteurs a été important pour la réussite de ce Projet 3. Une maîtrise complète des différents protocoles de communication a été nécessaire, tout comme le développement de codes de test spécifiques (par exemple pour le suivi des données des suiveurs de ligne en temps réel).

L'objectif a été atteint dans son intégralité.

8.4.1 Infrarouges DSL et DSR et extenseur PC8574.

Le PCF8574 est un circuit intégré, fonctionnant comme un extenseur d'entrées/sorties (E/S) à distance pour les bus bidirectionnels à deux fils (I²C). Ce dispositif permet d'ajouter efficacement 8 bits de ports quasi-bidirectionnels à un microcontrôleur via une interface série, minimisant ainsi le nombre de broches physiques requises sur le processeur central.

L'interface de communication repose sur le protocole I²C standard. Le PCF8574 se distingue par son adressage flexible grâce à trois broches d'adresse matérielles (A_0 , A_1 , A_2), permettant de configurer jusqu'à 8 adresses distinctes sur un même bus. En combinant cela avec la variante PCF8574A, qui possède une plage d'adresses différente, il est possible de connecter jusqu'à 16 dispositifs sur une seule ligne I²C, offrant ainsi un total de 128 ports d'E/S supplémentaires.

De plus, le composant intègre une sortie d'interruption active à l'état bas ($\overline{INT}$), laquelle se déclenche dès qu'un changement d'état survient sur l'un des ports d'entrée. Cette fonctionnalité est particulièrement importante pour les systèmes temps réel ou à faible consommation, car elle permet au microcontrôleur de rester en veille ou d'exécuter d'autres tâches sans avoir à effectuer un scrutage (polling) continu des entrées. Cependant, cette broche n'est pas connectée à notre Alphabot2-AR.

L'Alphabot2-AR utilise les ports P7 et P8 du PCF8574 pour recevoir les signaux infrarouges (DSL et DSR) situés sur les côtés de l'appareil à ultrasons. Le signal reçu via le protocole I²C de cet appareil est illustré sur la figure 19.

8.4.2 Configuration I2C et acquisition des capteurs infrarouges.

La configuration présentée initialise le périphérique "I2C1" du microcontrôleur STM32 en utilisant la bibliothèque HAL. Le bus I²C, comme nous l'avons vu pour le capteur SHT31, est un protocole de communication série synchrone utilisant deux lignes : SCL (horloge) et SDA (données).

La structure "I2C_HandleTypeDefhi2c1" contient tous les paramètres nécessaires à la

configuration du périphérique. La fonction "*MX_I2C1_Init()*" initialise ces paramètres et configure le matériel. Le paramètre "*Timing*" définit la fréquence et les timings du bus I²C (généralement calculés automatiquement par CubeIDE selon la fréquence souhaitée, par exemple 100 kHz ou 400 kHz). Ce registre contrôle les temps de montée, descente, et périodes d'horloge.

L'adresse propre "*OwnAddress1*" indique que le microcontrôleur n'est pas configuré ici comme esclave actif. Le mode d'adressage est en 7 bits ("*I2C_ADDRESSINGMODE_7BIT*"), ce qui est le standard le plus courant. Le mode double adresse est désactivé, donc une seule adresse est utilisée. L'appel général est désactivé, ce qui signifie que le périphérique ne répond pas aux adresses broadcast. Le mode "*No - Stretch*" est également désactivé, ce qui permet au périphérique de ralentir l'horloge si nécessaire.

Le filtre analogique est activé pour réduire le bruit sur les lignes, tandis que le filtre numérique est désactivé, donc aucun filtrage numérique supplémentaire n'est appliqué. Les broches PB8 et PB9 sont configurées respectivement comme SCL et SDA en mode fonction alternative open-drain avec résistances de pull-up, ce qui est requis pour le bus I²C. La vitesse des GPIO est réglée à très haute fréquence pour garantir de bonnes performances.

8.4.3 Logique pour contourner un obstacle.

La machine à états de la figure 12 décrit le comportement du robot pour le suivi de ligne et le contournement d'obstacles. En mode normal ("*LINE_FOLLOW*"), le robot suit la ligne à l'aide d'un contrôleur PD. Lorsqu'un obstacle est détecté via le capteur ultrasonique et les capteurs latéraux, il passe à l'état "*OBSTACLE_DETECTED*" où une rotation initiale est effectuée. Ensuite, dans "*LEAVE_OBSTACLE*", le robot contourne l'obstacle pendant un temps donné. Une fois cette phase terminée, il entre dans "*REALIGN*" afin de se repositionner vers la ligne. Si la ligne est détectée, il revient à "*LINE_FOLLOW*". Sinon, il passe en "*SEARCH_LINE*", où il effectue une rotation jusqu'à retrouver la ligne.

Dans les états "*OBSTACLE_DETECTED*", "*REALIGN*" et "*LEAVE_OBSTACLE*", nous avons ajouté une condition : si l'obstacle est toujours détecté après l'expiration du temps d'exécution de l'état, le robot s'en éloigne. Ceci garantit que l'algorithme quitte son état actuel et augmente la probabilité de revenir sur la ligne.

Il convient de noter que le comportement d'évitement d'obstacles repose principalement sur une approche empirique. Plusieurs essais expérimentaux ont été menés afin d'ajuster les durées des différentes phases (rotation initiale, contournement et réalignement) ainsi que les vitesses appliquées aux moteurs, dans le but d'obtenir un compromis satisfaisant entre réactivité et stabilité. Toutefois, cette stratégie ne repose pas sur un asservissement en boucle fermée vis-à-vis de l'obstacle, mais plutôt sur une séquence temporelle prédéfinie. Par conséquent, sa robustesse dépend fortement des conditions initiales et de l'environnement (position de l'obstacle, variations de surface, incertitudes mécaniques).

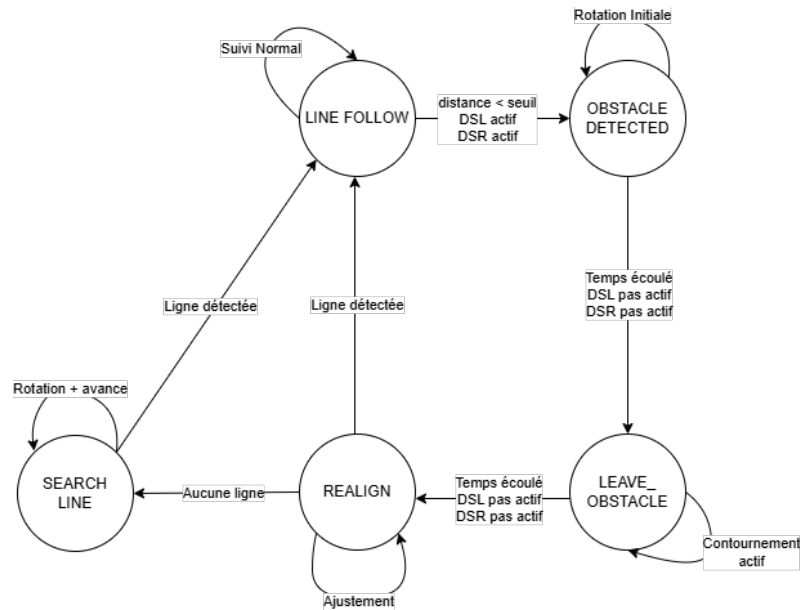


FIGURE 12 – Machine à états du comportement d'évitement d'obstacle et suivi de ligne.

Lors des expérimentations réalisées, il a été observé que le robot peut, dans certains cas, perdre définitivement la trajectoire après une manœuvre d'évitement. En pratique, ce phénomène s'est produit dans environ deux cas sur dix, sans possibilité de récupération autonome.

Ces résultats mettent en évidence les limites de l'approche actuelle et suggèrent qu'une amélioration future pourrait consister à intégrer une stratégie de perception plus avancée ou un contrôle adaptatif permettant une correction en temps réel durant l'évitement.

8.5 Remarques.

8.5.1 Inversion des broches du microcontrôleur.

Comme expliqué précédemment dans la section 8.3.1, les broches MISO et MOSI du protocole SPI sont inversées. Cette inversion nous contraint à utiliser des câbles (figure 13) pour établir la liaison entre la carte STM32 et les ports de l'alphabot2-AR. Cette contrainte a été identifiée en étudiant et en recoupant les différents documents techniques des nombreux participants.

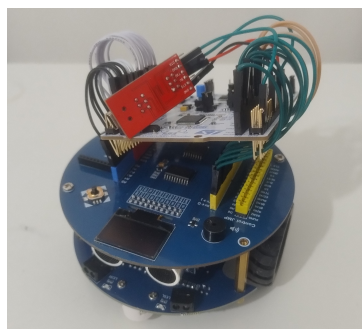


FIGURE 13 – Alphabot2 utilisé pendant le BE.

Plus précisément, les broches ADDR et DOUT de l'Alphabot2-AR doivent être connectées respectivement aux broches D11 et D12 du STM32. Toutefois, l'utilisation du shield introduit une inversion de ces deux lignes, comme il a été constaté expérimentalement. Cette inversion provoque un dysfonctionnement du protocole SPI, rendant la communication impossible sans correction matérielle ou logicielle.

8.5.2 Configuration du STM32 en mode autonome.

Pour faire fonctionner l'AlphaBot2 de manière autonome avec la carte Nucleo STM32L476RG, nous avons dû résoudre un problème lié aux cavaliers (jumpers) de la carte. Selon leur position, ces derniers autorisent ou non l'alimentation via la batterie de l'AlphaBot2. Pour un fonctionnement autonome, le cavalier doit être placé sur E5V. Cette difficulté a été l'une des plus complexes à identifier durant le projet.

8.5.3 Sensibilité des capteurs infrarouges à la lumière solaire.

Lors des différentes tentatives, nous avons constaté que les deux capteurs infrarouges situés à l'avant du véhicule étaient sensibles à la lumière du soleil. Comme c'est possible de le voir via la vidéo fournie en pièce jointe, on remarque en fonction de l'éclairage lumineux et de la position du robot par rapport au rayon du soleil les capteurs infrarouges détectent une présence ou non. Il est donc important d'utiliser le robot dans un éclairage artificiel et constant.

8.5.4 Interface série pour le débogage des données.

Afin de valider les informations obtenues via les protocoles SPI et I²C, l'interface série UART du STM32 a été utilisée pour afficher les données sur un terminal série, en l'occurrence PuTTY.

Dans ce but, une redéfinition de la fonction "`_write`" a été implémentée, permettant de rediriger les sorties standard (notamment "`printf`") vers l'interface UART. Cette approche facilite le débogage et la visualisation des données en temps réel.

La fonction utilisée est la suivante :

```
int _write(int file, char ptr, int len)
{
    HAL_UART_Transmit(&huart3, (uint8_t)ptr, len, HAL_MAX_DELAY);
    return len;
}
```

Cette implémentation permet d'envoyer directement les chaînes de caractères générées par "`printf`" via l'UART, assurant ainsi une interface simple et efficace pour le suivi du comportement du système.

8.6 Résultats.

8.6.1 Signaux du module ultrasonique.

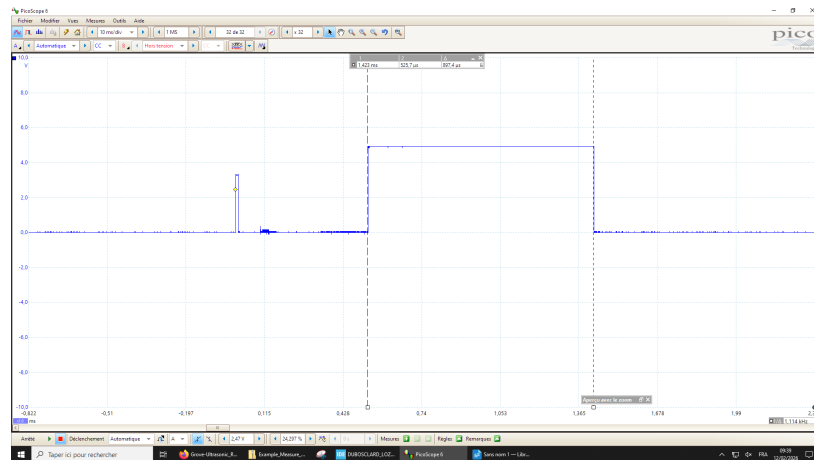


FIGURE 14 – Signaux d'ultrason.

La figure 14 montre l'envoi d'une impulsion de 10 microsecondes via la fonction "*delay_us*", expliquée dans la section 6. A la suite de la demande, le capteur va renvoyer une distance qui sera traduite et comparée à la distance minimale définie dans le code. Les réponses nous permettent de gérer la distance nécessaire pour pouvoir contourner un obstacle sur la ligne.

8.6.2 Signaux de commande des moteurs.

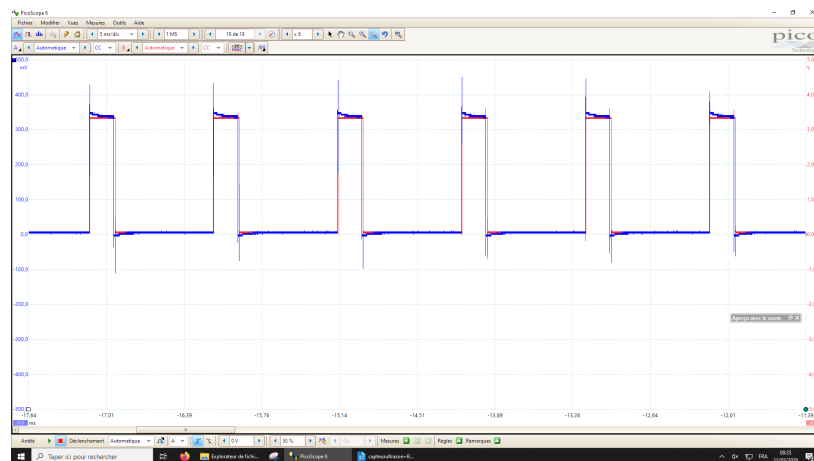


FIGURE 15 – Duty cycle du PWM utilisé pour les moteur droite et gauche.

Les signaux de commande des moteurs correspondent aux sorties PWM générées par le microcontrôleur et appliquées au driver TB6612FNG. Ces signaux permettent de contrôler la vitesse de rotation des moteurs.

Comme illustré sur la figure 15, lorsque les deux signaux PWM sont identiques, les moteurs tournent à la même vitesse, ce qui assure un déplacement rectiligne du robot.

8.6.3 Signaux des capteurs infrarouges IRx.

Sur la figure 16, on peut observer les informations envoyées par les capteurs (en bleu). On remarque l'apparition de six données alors qu'il n'existe que cinq capteurs. La toute première valeur n'est pas valide, car un capteur ne peut pas répondre instantanément à une requête : la demande est envoyée, mais la réponse ne figure que sur la trame suivante.

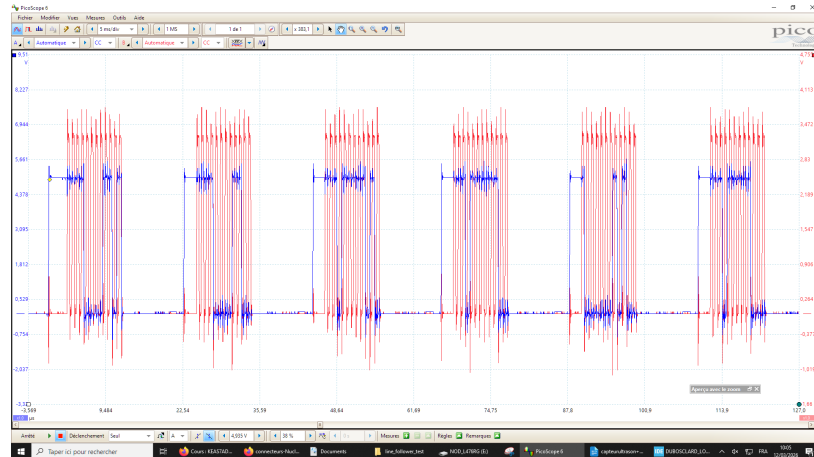


FIGURE 16 – Signaux envoyés par le TLC2543 avec le protocole SPI. Horloge en rouge, les données en bleu.

Ce fonctionnement engendre un décalage entre l'envoi de la requête et la réception des données pour chaque capteur. On note également que le capteur n°4 affiche une valeur différente des autres. Cela s'explique par le fait qu'au moment de l'acquisition, seul ce capteur se situait sur la ligne noire que l'AlphaBot2 devait suivre.

```
COM7 - PuTTY
IR: 3807 2626 648 2962 2790
IR: 3818 3038 642 3026 2866
IR: 3819 3448 634 3042 2960
IR: 3808 3394 544 2844 2922
IR: 3793 3228 510 2512 2818
IR: 3776 3142 510 2284 2780
IR: 3758 3112 572 2040 2776
IR: 3729 3072 622 1704 2768
IR: 3584 2932 609 1764 2656
IR: 3474 2840 552 1934 2572
IR: 3522 2834 422 2140 2632
IR: 3681 2912 454 2372 2776
```

FIGURE 17 – Données reçues du TLC2543 et affichées via une interface série.

Ces six données nous permettent de concevoir un suiveur de ligne pour l'AlphaBot2.

Chaque donnée correspond à une valeur numérique nous aidant à déterminer le seuil nécessaire pour que le robot suive la ligne sans aucune difficulté.

Afin d'effectuer des tests, nous avons ajouté des lignes permettant de lire les valeurs des capteurs en temps réel. Cela nous aide à comprendre comment ces derniers fonctionnent en pratique. Nous avons également réalisé des tests de vérification à l'aide de caches témoins, afin de tester les capteurs l'un après l'autre par exemple et de s'assurer de leur bon fonctionnement.

Sur la figure 17, on peut observer un exemple des informations reçues en temps réel via le logiciel PuTTY. Cette interface nous permet de lire les valeurs des cinq capteurs infrarouges situés sous le robot, afin de connaître les valeurs des couleurs et ainsi fixer les seuils du suiveur de ligne. Grâce à cette fenêtre, nous avons également pu confirmer le bon fonctionnement de chaque capteur.

8.6.4 Signaux des capteurs DSL et DSR.

Les signaux issus des capteurs infrarouges latéraux DSL et DSR sont acquis via l'expansur d'entrées/sorties PC8574 utilisant le protocole I²C. Ces capteurs permettent de détecter la présence d'obstacles à proximité immédiate du robot.

Les données récupérées correspondent à des niveaux logiques indiquant la détection ou non d'un obstacle. Une activation simultanée des deux capteurs indique la présence d'un obstacle en face du robot, tandis qu'une activation asymétrique permet d'identifier la direction relative de l'obstacle.

La figure 18 illustre l'activation simultanée des deux capteurs, tandis que la figure 19 présente les données correspondantes affichées sur l'interface série.

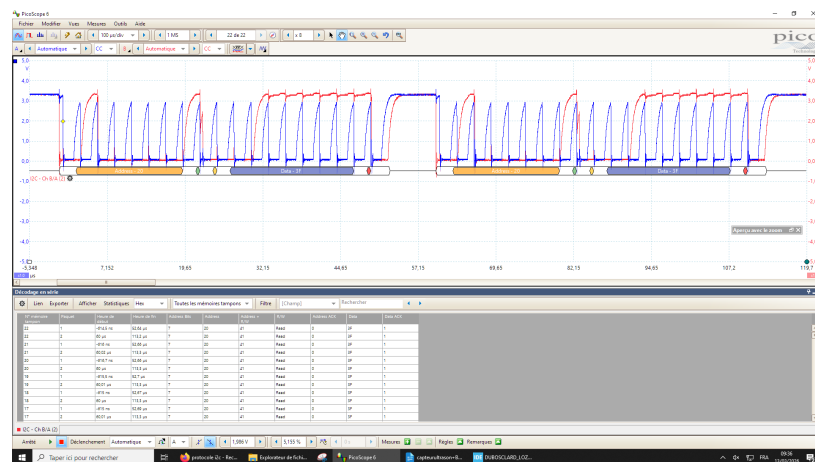
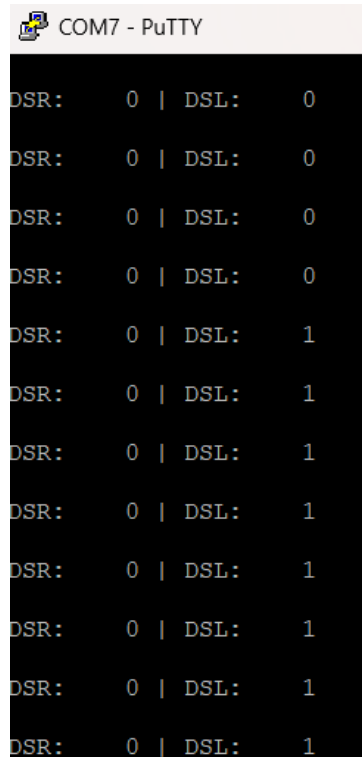


FIGURE 18 – Signaux envoyés par le TLC2543 avec le protocole . Deux capteurs actifs.



DSR:	0		DSL:	0
DSR:	0		DSL:	0
DSR:	0		DSL:	0
DSR:	0		DSL:	0
DSR:	0		DSL:	1
DSR:	0		DSL:	1
DSR:	0		DSL:	1
DSR:	0		DSL:	1
DSR:	0		DSL:	1
DSR:	0		DSL:	1
DSR:	0		DSL:	1
DSR:	0		DSL:	1
DSR:	0		DSL:	1

FIGURE 19 – Données reçues du PC8574 et affichées via une interface série.

9 Conclusion.

Ce projet nous a permis de mettre en pratique l'ensemble des notions fondamentales abordées en systèmes embarqués, en combinant de manière cohérente les aspects matériels et logiciels.

Dans un premier temps, nous avons acquis une maîtrise progressive de l'environnement STM32CubeIDE ainsi que de la configuration des principaux périphériques du microcontrôleur. L'étude et l'intégration de différents capteurs (ultrason, SHT31, infrarouges) ont permis de consolider notre compréhension des protocoles de communication série, notamment I²C et SPI, ainsi que des contraintes associées à leur mise en œuvre.

L'intégration de ces différents éléments au sein du robot AlphaBot2 a constitué une étape centrale du projet. Nous avons ainsi développé plusieurs fonctionnalités autonomes, telles que l'arrêt à distance, le suivi de ligne basé sur un correcteur PD, ainsi que l'évitement d'obstacles. Cette phase a mis en évidence l'importance de la structuration logicielle, de la gestion d'une machine à états et de la synchronisation entre les différents capteurs et actionneurs.

Toutefois, certaines limites ont été identifiées, en particulier concernant la robustesse de l'algorithme d'évitement d'obstacles, qui repose sur une approche empirique et temporelle. Une amélioration possible consisterait à mettre en place un asservissement en boucle fermée ou à intégrer des stratégies de contrôle plus avancées, permettant une adaptation en temps réel aux variations de l'environnement.

Enfin, ce projet a renforcé notre autonomie, notre capacité à exploiter une documentation technique complexe ainsi que notre aptitude à résoudre des problèmes concrets liés à l'intégration de systèmes embarqués. Il constitue ainsi une première approche complète du développement de systèmes autonomes, depuis l'analyse du cahier des charges jusqu'à la validation expérimentale.

Références

- [1] Waveshare, “Alphabot2-ar robot building kit for arduino.” <https://www.waveshare.com/alphabot2-ar.htm>, n.d. En línea; consultado : 2026.
- [2] Seeed Studio, “Grove – ultrasonic ranger wiki.” https://wiki.seeedstudio.com/Grove-Ultrasonic_Ranger/, n.d. En línea; consultado : 2026.
- [3] Mouser Electronics, “Tb6612fng datasheet.” <https://www.mouser.fr/datasheet/3/1500/1/F0D259E3DA939F7F8987E2CA6B88096BB915688DC84214F86CA6F77BB2B982D9.pdf>, n.d. En línea; consultado : 2026.
- [4] Texas Instruments, “Pcf8574 remote 8-bit i/o expander for i2c bus – datasheet.” <https://www.ti.com/lit/ds/symlink/pcf8574.pdf>, n.d. En línea; consultado : 2026.
- [5] Texas Instruments, “Tlc2543 12-bit analog-to-digital converter – datasheet.” <https://www.ti.com/lit/ds/symlink/tlc2543.pdf>, n.d. En línea; consultado : 2026.